Scanner Generalization for Mitosis Detection in MIDOG++

Prepared for the AI for Health project demo.

1. Project Goal

The goal of this project is to study whether a mitotic figure detection model can generalize across scanner types in real digital pathology images. This matters because pathology models may perform well on images from one scanner but degrade when tested on images from another scanner due to differences in color, stain appearance, contrast, sharpness, and image acquisition.

The working research question is:

Can stain/color augmentation improve scanner generalization for mitotic figure detection on MIDOG++ breast cancer pathology images?

2. Dataset Prepared

We prepared a breast cancer subset from the MIDOG++ data. The subset includes pathology image files and metadata/labels needed by teammates:

* Images: breast_cancer_subset/images/
* Slide metadata: breast_cancer_subset/metadata/breast_cancer_slides.csv
* Annotation JSON: breast_cancer_subset/metadata/annotations_breast_cancer.json
* Category file: breast_cancer_subset/metadata/categories.csv
* Annotation counts: breast_cancer_subset/metadata/annotation_counts_by_image.csv

The labels include mitotic figures and non-mitotic candidates. For detection training, the mitotic annotations were used as bounding boxes. Bounding boxes are represented in image coordinates.

Important scanner groups used in the experiments:

* Aperio CS2
* Hamamatsu S360
* Hamammatsu XR

3. Experimental Setup

We used a leave-one-scanner-out setup, abbreviated as LOSO. In LOSO, one scanner is held out for testing while the model trains on the other scanners. This directly tests scanner generalization.

Example:

* Train on Hamamatsu S360 + Hamammatsu XR, test on Aperio CS2.
* Train on Aperio CS2 + Hamammatsu XR, test on Hamamatsu S360.
* Train on Aperio CS2 + Hamamatsu S360, test on Hamammatsu XR.

This setup is more meaningful than a random split for this project because a random split may place images from the same scanner in both training and testing, which can overestimate generalization.

4. Patch-Level Baseline

Before training a full detector, we built a patch classification baseline. This used cropped patches around candidate regions and tested whether the model could classify mitotic vs non-mitotic regions across held-out scanner types.

Two patch models were compared:

* Pixel baseline: a small image-based classifier without color augmentation.
* Pixel augmented: the same general approach with stain/color augmentation.

Patch-level LOSO results:

Held-out scanner | Pixel F1 | Augmented F1 | Delta F1 | Pixel Recall | Augmented Recall
Aperio CS2 | 0.412 | 0.450 | +0.038 | 0.384 | 0.440
Hamamatsu S360 | 0.346 | 0.413 | +0.068 | 0.345 | 0.466
Hamammatsu XR | 0.529 | 0.525 | -0.004 | 0.794 | 0.785

Mean patch-level results:

Model | Mean F1 | Mean Precision | Mean Recall | Mean Accuracy
Pixel baseline | 0.429 | 0.395 | 0.508 | 0.509
Pixel augmented | 0.463 | 0.409 | 0.564 | 0.510

Interpretation:

Color augmentation improved mean held-out F1 from 0.429 to 0.463 and mean recall from 0.508 to 0.564. This supported the idea that stain/color variation may help scanner generalization, but this was still only a patch-level result.

5. Full Detection Dataset

To move closer to real pathology, we created full detection tile datasets. The original pathology images are too large to train on directly, so they were converted into 512 x 512 image tiles.

For each scanner group, tiles were generated with both positive and negative examples:

Scanner | Train tiles | Train positive | Train negative | Test tiles | Test positive | Test negative
Aperio CS2 | 2078 | 1033 | 1045 | 1376 | 688 | 688
Hamamatsu S360 | 2287 | 1139 | 1148 | 1164 | 582 | 582
Hamammatsu XR | 2549 | 1270 | 1279 | 902 | 451 | 451

This created a more realistic object detection task because the model must find mitotic figures in image tiles rather than classify pre-cropped candidates.

6. Full Detection Model

We trained Faster R-CNN with a ResNet-50 FPN backbone using PyTorch/Torchvision. The task was configured as a two-class detection problem:

* Background
* Mitotic figure

Two detector settings were compared:

* Baseline detector: no color jitter augmentation.
* Color-jitter detector: training images augmented with color/stain variation.

Evaluation used detection matching with IoU threshold 0.3 and multiple confidence score thresholds. For each checkpoint, the best score threshold was selected by F1.

7. Initial Detection Results and Issue Found

The first full detection runs showed that final checkpoints were unreliable. Some models had good intermediate checkpoints but collapsed or degraded by the final epoch.

Example:

* Hamammatsu XR color-jitter final checkpoint had F1 = 0.000.
* The same run at checkpoint epoch 3 had F1 = 0.630.

This means the method was not actually failing completely; rather, the last checkpoint was a poor representative of the model. Because of this, checkpoint sweeping was added.

8. Checkpoint-Sweep Detection Results

We evaluated every saved checkpoint and selected the best checkpoint per scanner/model.

Held-out scanner | Model | Best checkpoint | Score | Precision | Recall | F1
Aperio CS2 | Baseline | epoch 4 | 0.700 | 0.465 | 0.654 | 0.544
Aperio CS2 | Color jitter | epoch 5 | 0.700 | 0.604 | 0.727 | 0.660
Hamamatsu S360 | Baseline | epoch 4 | 0.700 | 0.650 | 0.650 | 0.650

Hamamatsu S360 | Color jitter | epoch 3 | 0.700 | 0.207 | 0.761 | 0.326
Hamammatsu XR | Baseline | epoch 3 | 0.700 | 0.623 | 0.491 | 0.550
Hamammatsu XR | Color jitter | epoch 3 | 0.700 | 0.596 | 0.669 | 0.630

Mean best-checkpoint results:

Model | Mean Precision | Mean Recall | Mean F1
Baseline | 0.580 | 0.598 | 0.581
Color jitter | 0.469 | 0.719 | 0.539

9. Interpretation

The patch-level experiment suggested that color augmentation can improve scanner generalization, especially recall. The full detection experiment showed a more complicated but more realistic picture.

Key findings:

1. Checkpoint choice matters. Several final checkpoints were worse than earlier checkpoints. This suggests training instability and possible overfitting.

2. Color jitter increased recall overall. The color-jitter detector had mean recall 0.719 compared with 0.598 for the baseline. This means it found more true mitotic figures.

3. Color jitter reduced precision overall. The color-jitter detector had mean precision 0.469 compared with 0.580 for the baseline. This means it produced more false positives.

4. Color jitter helped some scanner holdouts but hurt others. It improved Aperio CS2 and Hamammatsu XR, but performed poorly on Hamamatsu S360.

5. Hamamatsu S360 appears to be a difficult or mismatched scanner domain for the current augmentation approach. The S360 color-jitter model had high recall but many false positives, which caused low F1.

10. Current Conclusion

The current results support the broader project idea that scanner appearance affects mitosis detection performance. However, simple color jitter is not a complete solution.

The strongest current conclusion is:

Scanner generalization is sensitive to both augmentation strategy and training stability. Color augmentation can improve recall and help some held-out scanner domains, but it can also increase false positives and reduce F1 if the augmentation does not match the held-out scanner distribution.

11. Next Steps

The next experiment is to rerun Faster R-CNN with more stable training settings:

* Lower learning rate: 1e-4 instead of 5e-4.
* Gradient clipping: max norm 1.0.
* More epochs with best-checkpoint selection.
* Separate output folders so older results are preserved.

After that, the recommended next comparisons are:

* Compare best-checkpoint Faster R-CNN against a detector such as RetinaNet.
* Try more pathology-specific stain augmentation or stain normalization.
* Analyze false positives by scanner to understand what visual patterns the detector confuses with mitotic figures.
* Consider adversarial/domain adaptation after establishing a stable detector baseline.

12. Limitations

These results are based on the prepared MIDOG++ breast cancer subset and scanner holdout splits used for the demo. They should not be interpreted as final challenge-test performance. The hidden or official challenge test set was not used here. The current work is best understood as a controlled project demo showing scanner-domain generalization issues and possible mitigation strategies.